

Gestione delle tappe tramite git tags

L'idea generale è di utilizzare le tag di git per identificare alcuni specifici commit nella storia di git come commit corrispondenti alle tappe. Questo fa sì che le tappe siano estraibili in una cartella ignorata da git, e poi compilabili facendo un ciclo sulle sottocartelle corrispondenti.

Questo vi permette di lavorare sul progetto come se fosse un progetto git normale: la storia del progetto è rappresentata dai commit, come dovrebbe essere, e il vostro `CMakeLists.txt` compila solo l'eseguibile del vostro progetto, usando i file sorgente del commit in quel momento attivo nel repo git.

Nel repository non ci saranno quindi duplicazioni di codice nelle sottocartelle delle tappe committati in contemporanea, e non bisogna modificare il `CMakeLists.txt` se non quando aggiungete nuove file sorgente `cpp` o `cc`, nuove librerie o altro.

Bisogna solo ricordarsi di creare una tappa nei momenti opportuni, usando il comando `git tag`, o il comando `./stages.sh tag` dello script che vi fornisco.

Workflow semi-manuale

- Dopo aver fatto un commit che è un buon candidato per una tappa, si può usare il comando
`git tag -a Stage_NN -m 'descrizione della tappa'`
per creare una tag annotata con un messaggio di descrizione, chiamata `Stage_NN`, dove `NN` è il numero della tappa.
- Avvertimento ovvio: il comando `tag` crea una tag del commit corrente, ignora completamente qualsiasi modifica non committata possiate avere nella working directory.
- I file tracciati in quel commit (evitando quindi di spostare le cose ignorate da git) possono venire copiati in una sottocartella, ad esempio `./FCG_Stages/Stage_NN`, dentro al repository stesso. La cartella `./FCG_Stages` deve venire ignorata da git, visto che le tappe lì dentro sono solo una copia dei file nei commit, e possono venire nuovamente estratti in qualsiasi momento con il comando `git checkout Stage_NN` seguito da una copia dei file.
- Per rispondere ai requisiti di consegna, basterebbe un piccolo script che faccia loop su tutte le tag corrispondenti alle tappe per:
 - Estrarre tutte le tappe (visto che la copia è ignorata da git).
 - Chiamare `cmake` per configurare il build di ogni tappa.
 - Chiamare `cmake` per compilare ogni tappa.

Detto questo, vi fornisco uno script più completo, che automatizza il più possibile la gestione dell'intero workflow, incluso il tagging, la copia dei file, estrazione, compilazione, e qualche funzionalità aggiuntiva.

Workflow automatico tramite script `stages.sh`

Innanzitutto, scaricare `stages.sh`, inserirlo nella root del repository del progetto git, renderlo eseguibile tramite `chmod +x stages.sh`, e committarlo con git. Le operazioni possibili sono:

- **Creazione** di una tappa al commit attuale, con descrizione, e numero assegnato incrementalmente in modo automatico
`./stages.sh tag -d 'qui descrizione della tappa'`
- **Export** di tutte le tappe nella cartella `FCG_Stages`
`./stages.sh export`
- **Build e compilazione** di tutte le tappe
`./stages.sh compile`
- **Export + Compilazione** insieme
`./stages.sh all`
- **Stampa** di tutte le tappe presenti nelle tag, con descrizione
`./stages.sh print`
- **Rimozione** della tappa più recente (se vi rendete conto che non compila, o che sarebbe meglio sostituirla con un commit successivo)
`./stages.sh remove`
- **Aggiornamento** di una tappa specifica numero XX con una nuova descrizione
`./stages.sh -n XX -d 'new description'`

Lo script permette di usare la prima lettera dei comandi come abbreviazione, modifica autonomamente `.gitignore` quando viene lanciato la prima volta, e ha un comando `help` da usare come riferimento veloce:

Usage: `./stages.sh {t|x|c|a|p|r|u|h}` [arguments]

Commands:

t tag -d "description"	Create a new stage tag with description
x export	Export all stages to FCG_Stages directory
c compile	Compile all stages in FCG_Stages directory
a all	Export and compile all stages at once
p print	Print all stage numbers, with description
r remove	Remove last stage
u update -n NUM -d "description"	Update the description of a stage
h help	Print usage and exit

Istruzioni di consegna se integrate lo script nel vostro workflow

- Estrae tutte le tappe.
- Controllate che compilino senza errori.
- Controllate che funzionino come previsto.
- Se usate un servizio git su cloud, basta che il repo remoto sia aggiornabile, e ci mandate il link in un file `.txt` che uploadate.

- Se volete consegnare uno zip, rimuovete la cartella `build` e la cartella `FCG_Stages`, controllate che non ci siano file non tracciati o modifiche non committate, comprimate la cartella di progetto (quella che contiene la directory nascosta `.git`) e la consegnate.